

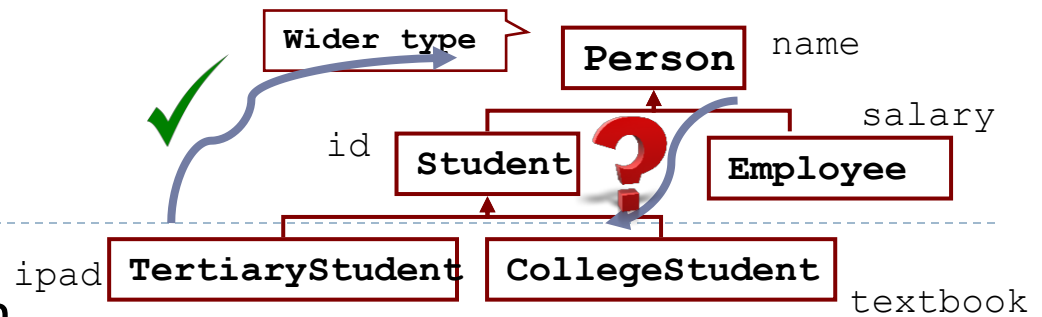


Object Oriented Software Development

Abstract Classes



Review



► Casting from Student to Person

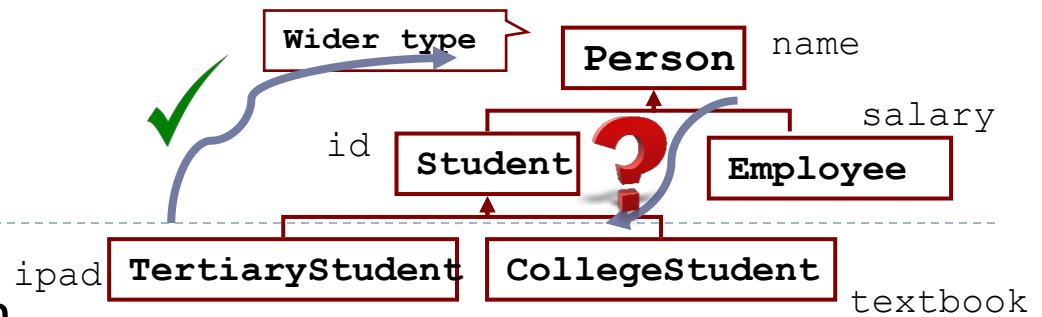
- Student s = new Student(); ✓ //s.id ✓ s.name ✓
- Person p = (Person) s; //p.name p.id

► Casting from Person to Student ?

- Person p1 = new Person(...);
- Person p2 = new Student(...);
- Person p3 = new TertiaryStudent(...);
- Person p4 = new Employee(...);
- Student s1 = (Student) p1; //CT: //RT: (no id at all)
- Student s2 = (Student) p2; //CT: //RT:
- Student s3 = (Student) p3; //CT: //RT: s3.id p3.id
- Employee e1 = (Employee) p1; //CT: //RT:
- Employee e2 = (Employee) s; //CT: //RT:
- Employee e3 = (Employee) p4; //CT: //RT:



Review



► Casting from Student to Person

- Student s = new Student(); ✓ //s.l.id ✓ s.l.name ✓
- Person p = (Person) s; ✓ //p.name ✓ p.id ✗

► Casting from Person to Student ?

- Person p1 = new Person(...);
- Person p2 = new Student(...);
- Person p3 = new TertiaryStudent(...);
- Person p4 = new Employee(...);
- Student s1 = (Student) p1; //CT: ✓ //RT: ✗ (no id at all)
- Student s2 = (Student) p2; //CT: ✓ //RT: ✓
- Student s3 = (Student) p3; //CT: ✓ //RT: ✓ s3.id ✓ p3.id ✗
- Employee e1 = (Employee) p1; //CT: ✓ //RT: ✗
- Employee e2 = (Employee) s; //CT: ✗ //RT: ✗
- Employee e3 = (Employee) p4; //CT: ✓ //RT: ✓



Agenda & Reading

► Topics:

- Introduction
- Abstract Classes
- Interfaces
- Abstract Classes VS Interfaces
- Extending Interfaces
- Final variables, methods, classes

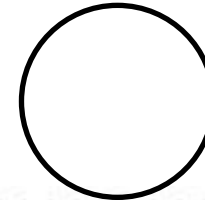
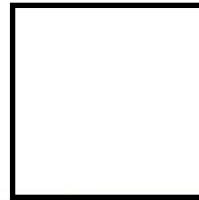
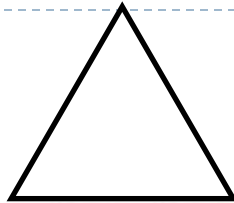
► Reading

- Java how to program Late objects version (D & D)
 - Chapter 10
- The Java Tutorial:
 - Abstract Methods and Classes
 - <http://docs.oracle.com/javase/tutorial/java/landl/abstract.html>
 - Writing Final Classes and Methods
 - <http://docs.oracle.com/javase/tutorial/java/landl/final.html>
 - Interfaces
 - <http://docs.oracle.com/javase/tutorial/java/landl/createinterface.html>

1.Introduction

Example: ShapesDemo.java

Design 1: Triangle, Rectangle ...



```
Triangle
- p : Point
- width : int
- height : int
+ Triangle(x : int, y : int, w : int, h : int)
+ getX() : int
+ getY() : int
+ getWidth() : int
+ getHeight() : int
+ setX(x : int) : void
+ setY(y : int) : void
+ setWidth(w : int) : void
+ setHeight(h : int) : void
+ translate(deltaX : int, deltaY : int) : void
+ draw(g : Graphics) : void
+ area() : int
```

```
Rectangle
- p : Point
- width : int
- height : int
+ Rectangle(x : int, y : int, w : int, h : int)
+ getX() : int
+ getY() : int
+ getWidth() : int
+ getHeight() : int
+ setX(x : int) : void
+ setY(y : int) : void
+ setWidth(w : int) : void
+ setHeight(h : int) : void
+ translate(deltaX : int, deltaY : int) : void
+ draw(g : Graphics) : void
+ area() : int
```

```
Circle
- p : Point
- radius : int
+ Circle(x : int, y : int, r : int)
+ getX() : int
+ getY() : int
+ getRadius() : int
+ setRadius(r : int) : void
+ setX(x : int) : void
+ setY(y : int) : void
+ translate(deltaX : int, deltaY : int) : void
+ draw(g : Graphics) : void
+ area() : int
```

repeating

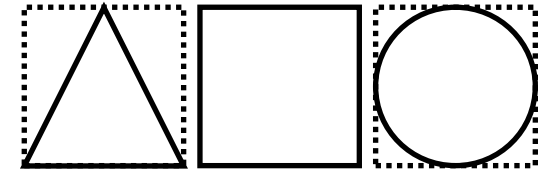
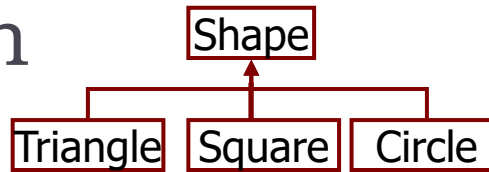
- ▶ General methods:
 - ▶ get(), getY(), setX(...), setY(...), translate(...)
- ▶ Specific methods: (need to know the type)
 - ▶ draw(...), getArea()



Bad design!



1. Introduction Inheritance



▶ Superclass: Shape

▶ Common

- ▶ Instance variables: Point p (top left corner)
- ▶ Methods: set, get and translate (common to all shapes)

▶ Specific methods : draw, area

- ▶ Need to draw/calculate an area for all shapes
- ▶ But this is done differently for different shapes

▶ Subclasses: Triangle, Square, Circle

▶ Use an array of Shapes

- ▶ Create an array to store different shapes

▶ Invoke the draw method in the array of shapes

- ▶ But how do you implement the draw/area method in the superclass?

- ▶ It might not be appropriate to have any code for the getArea() method in the Shape class at all (you don't know how to calculate the area of an object if you don't know the type of the shape).

```
public class Shape {  
    protected Point p; //top-left corner  
    public int getX() { return p.x; }  
    ...  
}
```

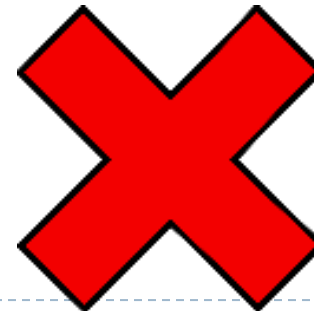
```
Shape[] s = new Shape[3];  
s[0] = new Rectangle(...);  
...  
for (int i = 0; i < s.length; i++) {  
    s[i].draw(g);  
}
```

Create an array

Invoke the draw method

```
public class Shape {  
    ...  
    public void draw(Graphics g) {??????}  
    public int getArea() {??????}  
    ...  
}
```

Need to Change!





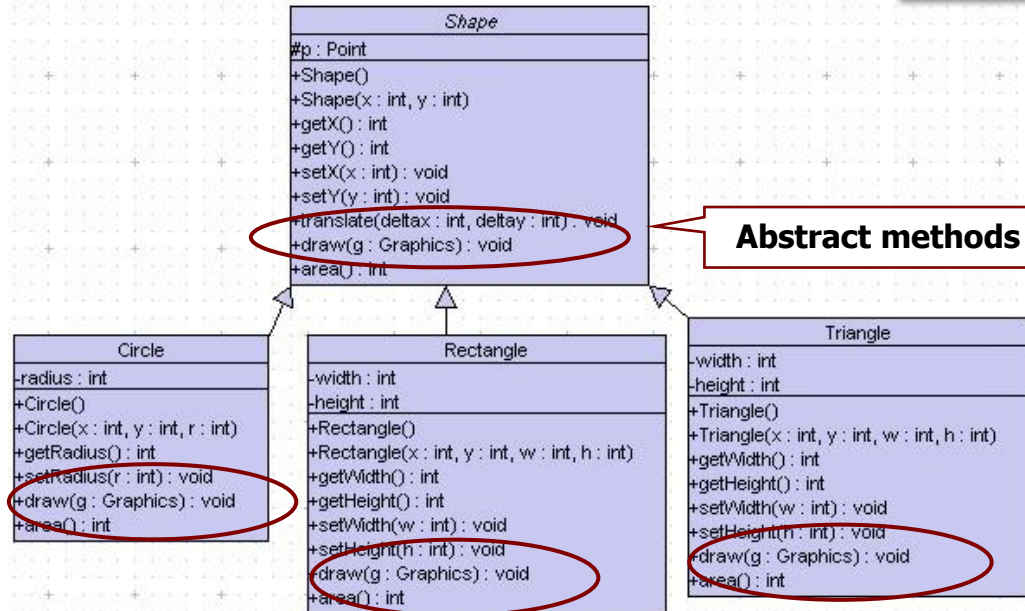
1.Introduction

Abstract classes

```
public abstract class Shape {  
    ...  
    public abstract void draw(Graphics g);  
    public abstract int getArea();  
}
```

Remove the code inside the `getArea()` method and make the method abstract

Example: `abstract/ShapesDemo.java`



Provide implementation in subclasses

```
public class Rectangle extends Shape {  
    private int width, height;  
    public int getArea() {  
        return (width * height);  
    }  
    ...  
}
```

```
public class Circle extends Shape {  
    private int radius;  
    public int getArea() {  
        return (int) (Math.PI * radius * radius);  
    }  
    ...  
}
```

```
public class Triangle extends Shape {  
    private int width, height;  
    public int getArea() {  
        return (width * height) / 2;  
    }  
    ...  
}
```



2. Abstract classes

Concrete & Abstract classes

► Concrete classes

- Classes that can be used to **instantiate** objects are called concrete classes.
- Such classes provide implementations of **every** method they declare (some of the implementations can be inherited).

► Abstract classes

- Used only as superclasses in **inheritance hierarchies**, so they are sometimes called abstract superclasses.
- Cannot be used to instantiate objects—abstract classes are incomplete and they specify only what is common among subclasses.
- Subclasses must declare the “missing pieces” to become “concrete” classes, from which you can instantiate objects; otherwise, these subclasses, too, will be abstract.

```
Shape s = new Shape(); <- wrong
```




2. Abstract Classes

Abstract Methods

- ▶ Abstract classes permit declaration of classes that define only part of an implementation, leaving the subclasses to provide the details:
 - ▶ Implementation of **common state** and **behaviour** that will be inherited by subclasses
 - ▶ **Declaration** of **generic** behaviours that subclasses must implement
- ▶ You make a class abstract by declaring it with keyword **abstract**.

```
public abstract class Shape {  
    ...
```
- ▶ An abstract class normally contains one or more abstract methods.
 - ▶ An abstract method is an instance method with keyword **abstract** in its declaration, as shown below:

```
public abstract void draw(Graphics g) ;  
public abstract int getArea() ;
```
 - ▶ Abstract methods do not provide implementations



2. Abstract Classes Rules

- ▶ An abstract class is not required to have an abstract method in it.
 - ▶ But a class that contains abstract methods must be an abstract class even if that class contains some concrete (non-abstract) methods.
 - ▶ When a class inherits from an abstract class, it must implement every abstract member defined by the abstract class.
 - ▶ Failure to implement a superclass's abstract method in a subclass is a compilation error unless the subclass is also declared abstract.
 - ▶ Constructors and static methods cannot be declared abstract.
 - ▶ An abstract class cannot be instantiated, however references to an abstract class can be declared.
 - ▶ These can hold references to objects of any concrete class derived from those abstract superclasses.
-

Exercise 1-2

- ▶ Consider the Shape, Rectangle and Circle classes defined previously:
 - ▶ Add an abstract `getPerimeter()` method into the Shape class
 - ▶ Implement the `getPerimeter()` in all the subclasses.

```
public abstract class Shape {  
    ...  
    public abstract void draw(Graphics g);  
    public abstract double ...  
}
```

```
class Rectangle extends Shape {  
    private int width, height;  
    ...  
}
```

```
class Circle extends Shape {  
    private int radius;  
    ...  
}
```



Exercise 3

- ▶ Define an abstract class named **MyBase** which contains two abstract methods:
 - ▶ `int method1(int a, int b);`
 - ▶ `int method2(int a, int b);`
- ▶ Define a subclass named **MyDerived** which extends the abstract class **MyBase**
 - ▶ `method1()` returns the sum of parameter `a` and `b`
 - ▶ `method2()` returns the product of parameter `a` and `b`.

```
MyDerived b = new MyDerived();  
System.out.println(b.method1(5, 2));  
System.out.println(b.method2(5, 4));
```

7
20



Exercise 4

- ▶ Define an abstract class named **Animal** which contains a abstract method:
 - ▶ `public void makeSound();`
- ▶ Define a subclass named **Dog** which extends the abstract class **Animal**. The `makeSound()` method prints "Woof"
- ▶ Define a subclass named **Cat** which extends the abstract class **Animal**. The `makeSound()` method prints "Meow"

```
Animal a = new Cat();  
a.makeSound();  
Animal b = new Dog();  
b.makeSound();
```

```
Woof  
Meow
```



Exercise 5

- ▶ Define an abstract class named `Person` which represents a `Person` object. The class contains the following:
 - ▶ `name`
 - ▶ `constructor`
 - ▶ `getName()`, `setName()`
 - ▶ `abstract void eat()`
 - ▶ `toString()`

```
Student s = new Student("Michael");  
s.eat();  
System.out.println(s);
```

```
eating fried rice!  
Student(Michael)
```



Exercise 6-9

- ▶ Continuing on from the previous question, define the following subclasses:
 - ▶ Student: eating fried rice!
 - ▶ CollegeStudent: eating Subway Sub of the Day!
 - ▶ TertiaryStudent: eating roast chicken!
 - ▶ Employee: eating buffet!

```
Person[] people = new Person[4];  
people[0] = new Student("Michael");  
people[1] = new CollegeStudent("Paul");  
people[2] = new  
TertiaryStudent("Peter");  
people[3] = new Employee("Anna");  
for (Person p: people) p.eat();
```

```
eating fried rice!  
eating Subway Sub of the Day!  
eating roast chicken!  
eating buffet!
```



Exercise 10

- ▶ Continuing on from the previous question, define a static method named `printEatMessages(Person[] array)`
 - ▶ prints a message from each object in the parameter array as shown in the following example:

```
public static void  
  
}
```

```
Person[] people = new Person[4];  
people[0] = new Student("Michael");  
people[1] = new CollegeStudent("Paul");  
people[2] = new TertiaryStudent("Peter");  
people[3] = new Employee("Anna");  
printEatMessages(people)
```

```
Michael, eating fried rice!  
Paul, eating Subway Sub of the Day!  
Peter, eating roast chicken!  
Anna, eating buffet!
```